

AR Washing Machine Viewer

Project Plan & Technology Guide — view an IFB washing machine in your own room, straight from the web browser (no app).

The one-line summary

We build the **Shopify / Amazon “View in your room”** experience, **browser-only**, using Google’s **<model-viewer>** with a **GLB** model (plus a **USDZ** copy for iPhone). One web page works on Android, iPhone, tablet and laptop — the machine sits on the real floor at true size, shows its dimensions, and runs smoothly on the phone’s own AR engine (ARCore / ARKit).

Prepared for	IFB — IT / AR Project	Approach	Web-AR (model-viewer)
Date	29 August 2026	Delivery	Standalone microsite + QR
Object	Washing machine (GLB)	Devices	Android · iPhone · Tablet · Laptop

Contents: 1) What we’re building 2) The 3 AR engines 3) What real apps use 4) Our decision 5) Model formats 6) Getting a good model 7) The sketch 8) The algorithm 9) Tech stack 10) Milestones 11) Blockers & risks 12) Trends & sources

SECTION 1

What we’re building

Imagine a customer looking at an IFB washing machine online. Instead of guessing whether it fits their laundry space, they open a web link on their phone, point the camera at the floor, and a **full-size 3D washing machine appears in their room**. They can walk around it, see it from every angle, read its exact dimensions, and decide with confidence — all **without installing any app**.

This is called **Web-AR** (augmented reality in the browser). It is exactly the “View in your room” feature you already see on Amazon and Shopify stores. Our job is to build one small, fast web page (a “microsite”) that does this for the washing machine.

What “good” means for this project (your requirements)

- **Browser-only** — no app download, works from a shared link or QR code.
- **Sits on the surface** — the machine rests flat on the real floor, anchored, never floating or sinking.
- **Shows dimensions** — Height × Width × Depth visible on screen and in AR.
- **Smooth** — no lag, no glitch, stable tracking, easy to position.
- **Every device** — Android, iPhone, tablet and laptop.

Scope: what we build first vs later

Phase	What it includes
MVP (first)	One washing-machine model, live 3D on all devices, “View in your space” AR on phones, dimensions readout, deploy to an HTTPS link + QR.
Later	Real IFB-scanned model, multiple products/variants, colours, analytics, embedding into IFB’s product pages.

SECTION 2

The 3 AR-engine options (and what each really means)

An “AR engine” is the software that shows the 3D model through the camera and makes it stick to the real world. There are three realistic routes for a web project. Here they are in plain language.

	<model-viewer> (Google)	Custom WebXR + Three.js	8th Wall (Niantic, now open-source)
What it is	One ready-made HTML tag	You hand-build the AR app in code	A full Web-AR engine with its own tracking
Setup effort	Very low — minutes	High	Medium–High (self-host)
Cost	Free	Free	Free now (open-sourced Feb 2026)
Android AR	Yes — WebXR / Scene Viewer	Yes — WebXR	Yes
iPhone AR	Yes — via Quick Look (USDZ)	No in-browser AR on iPhone	Yes (own tracking)
Laptop / desktop	Yes — 3D viewer fallback	Yes (you build it)	Yes
Tracking quality	Good (phone’s own AR)	Good (phone’s own AR)	Best — markerless SLAM + occlusion
Control of the UI	Limited (but enough)	Full	Full
Best for	Placing a product in a room	Custom / interactive AR apps	Filters, portals, games, marketing
Fit for us	Best fit	Overkill + no iPhone AR	Overkill for one appliance

In one line each:

- **<model-viewer>** — the easy, standard way to put a 3D product in AR. You drop in one tag and it automatically uses Android’s and iPhone’s built-in AR. *Catch:* you get a clean but somewhat fixed UI.
- **Custom WebXR + Three.js** — you code your own AR from scratch for total control. *Catch:* lots of work, and iPhone browsers still can’t do live in-page AR, so you’d fall back to Quick Look anyway.
- **8th Wall** — the most powerful tracking (it can even hide the model behind real objects). *Catch:* heavier setup and far more than we need to place one washing machine on a floor.

SECTION 3

What the well-known apps actually use

It helps to copy what already works at scale. Here is the honest breakdown:

App / experience	How AR works	Engine / tech
IKEA Place	Native app (download)	Apple ARKit (+ ARCore on Android)
IKEA Kreativ	Native app	ARKit + LiDAR room-scan + AI room design
Amazon — “View in Your Room”	Web + app	Web-AR with GLB + USDZ
Shopify AR (stores)	Browser, no app	GLB + USDZ via model-viewer / Quick Look
Google “View in 3D/AR” (Search)	Browser, no app	Scene Viewer + model-viewer
Nike / face filters / portals	Browser, no app	8th Wall (SLAM Web-AR)

The takeaway

The big furniture apps (IKEA) are **native apps on ARKit** — great, but they force an app download. For placing a product **in a room on the web**, the proven industry pattern is **GLB + USDZ through model-viewer / Quick Look** — the **same path Shopify and Amazon use**. That is precisely our need, so that is what we build.

Note: IKEA Kreativ also scans your room with LiDAR to build a 3D copy of it. That is a much bigger, native-app project. For our goal — letting a customer check whether a washing machine fits — the simpler Web-AR placement is the right size of solution.

SECTION 4

Our decision

- Decision: build on Google’s <model-viewer> (browser-only Web-AR)**
- **One tag, all four devices** — Android, iPhone, tablet and laptop are covered automatically.
 - **Free & open-source**, actively maintained by Google.
 - **The Shopify / Amazon-proven pattern** — low risk, well documented.
 - **Fast to ship** — minimal code means we get a working demo quickly.
 - **Easy to upgrade** — swapping in the real IFB model later is a one-line change.
 - If we ever need film-grade realism (hiding the model behind real furniture), we can add **8th Wall** later.

Every requirement you set maps cleanly onto this choice:

Your requirement	How model-viewer delivers it
Browser-only, no app	Runs as a normal web page; AR launches from a button.
Sits on the surface, anchored	Uses the phone’s own floor detection (ARCore/ARKit) via ar-placement=“floor” .
Correct size, no drift	ar-scale=“fixed” + a model built at real metres (1 unit = 1 m).
Show dimensions	Dimension “hotspots” read the model’s bounding box and print H×W×D.
No lag / glitch	Native OS tracking + a light, compressed model (Draco/meshopt).

SECTION 5

Model formats — does the format matter?

You asked whether switching format would make the model look better, or if all formats behave the same. Short answer: for our pipeline you only need two, and **format is not what makes a model look good**.

Format	Use it for	Notes
GLB	The master file — web + Android	Binary glTF: compact, supports PBR materials, the web standard. This is your main file.
USDZ	iPhone / iPad AR (Quick Look)	Required by Apple. We generate it <i>from</i> the GLB.
glTF (.gltf)	Editing / source	Same standard as GLB but split into files; pack to GLB for delivery.
FBX / OBJ / STL / 3DS	Not for web-AR	Editing/interchange formats — you convert them to GLB anyway.

Myth-buster: a new format will NOT fix a low-quality model

Your current GLB is an open-source rip and looks rough. Converting it to USDZ, FBX or anything else will **not** improve it. Visual quality comes from three things: **(1) geometry / topology** (how clean the 3D shape is), **(2) PBR textures** (realistic colour, metal, roughness maps), and **(3) correct real-world scale** (glTF: **1 unit = 1 metre**). Fix the *model*, not the container.

Recommendation: keep **GLB as the single master**, auto-generate a **USDZ** for iPhone, and if the look isn't good enough, replace the *model* (next section) rather than chasing formats.

SECTION 6

Getting a good washing-machine model

A) Placeholder to start today (free)

- **Sketchfab** — search “washing machine”, filter **Downloadable** + a **CC licence**, export as GLB. Good enough to build and test the whole app.
- **Khronos glTF Sample Assets** — clean reference models to verify the pipeline works.
- **model-viewer demo assets** — use any sample GLB purely to prove the AR flow before your real model is ready.

B) For the real, IFB-accurate model (production)

- **Scan the actual IFB unit** with **Polycam** or **Luma AI** on a phone (LiDAR / photogrammetry) — gives the true shape *and* correct dimensions. Best realism for least effort.
- **AI-generate** from product photos with **Meshy** (image → 3D), then clean up in Blender.
- **Convert IFB CAD** in Blender and export an optimized GLB (best if engineering files exist).

Whichever model you use, always do these three things

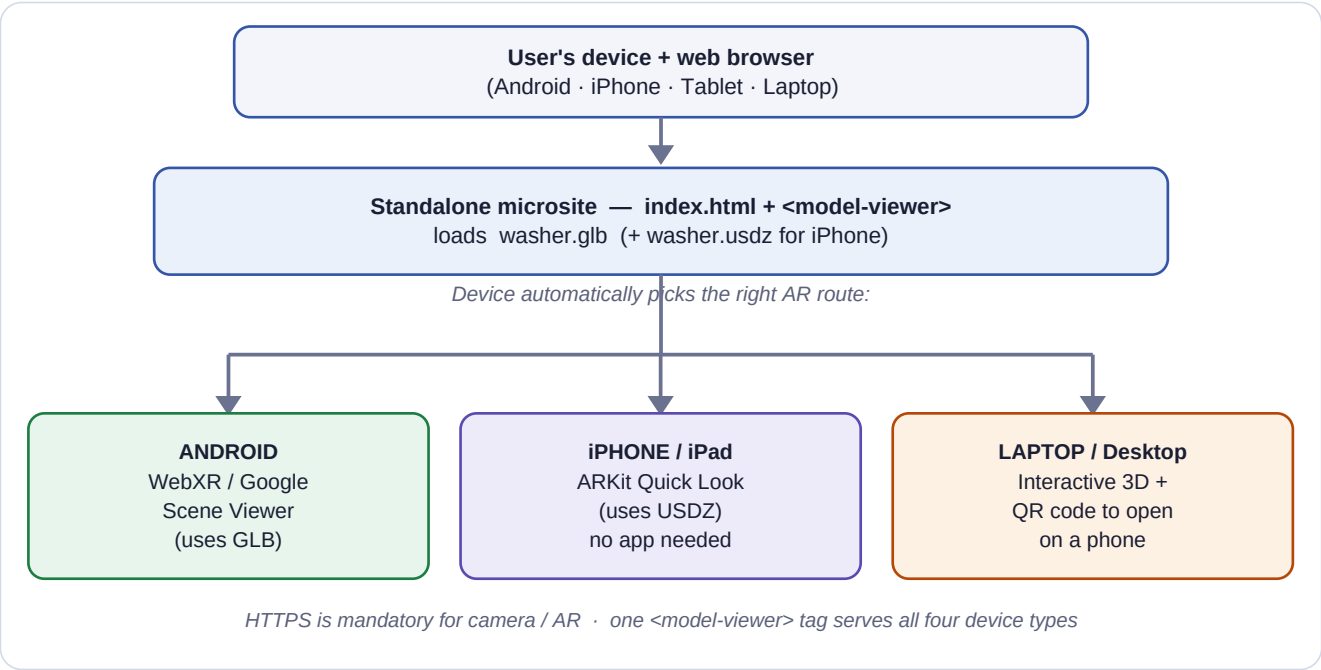
- Set the **origin/pivot at the base-centre** so it rests *on* the floor (not sunk half-in).
- Set **real-world size** (e.g. ~85 cm tall) so AR shows it at true scale.
- **Optimize**: bake PBR textures + compress with Draco/meshopt, target under ~5–10 MB for fast phone loading.

You stay in control: if you don't like the placeholder, dropping in your own GLB is literally a one-line change in the page.

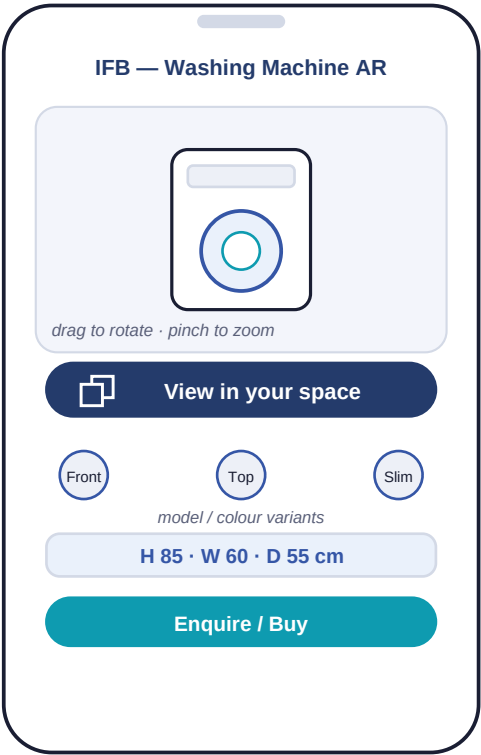
SECTION 7

The sketch

7.1 How the pieces fit together (architecture)



7.2 What the screen looks like (UI wireframe)



- 1 **Live 3D preview**
drag to rotate, pinch to zoom — works on every device.
- 2 **One tap → AR**
“View in your space” opens the camera and drops the machine on your floor.
- 3 **Variants**
switch model / colour without reloading.
- 4 **Real dimensions**
H × W × D shown on-screen and as AR labels.
- 5 **Call to action**
enquire, buy, or share the link.
- 6 **Laptop fallback**
shows 3D + a QR code to continue on a phone.

SECTION 8

The algorithm (step-by-step flow)

This is what happens, in order, from opening the link to placing the machine:



The tag that makes it all work

```
<model-viewer src="washer.glb" ios-src="washer.usdz" ar ar-modes="webxr scene-viewer quick-look" ar-placement="floor" ar-scale="fixed" camera-controls shadow-intensity="1">
```

One element: it shows live 3D everywhere, and on a phone the **AR button** hands off to the device's own AR engine, which finds the floor and anchors the machine at real size.

SECTION 9

The tech stack (plain and simple)

Layer	Tool	Why
3D / AR viewer	Google <model-viewer>	Free, one tag, works on all devices
Master format	GLB	The web + Android standard
iPhone format	USDZ (made from the GLB)	Required by Apple Quick Look
GLB → USDZ	Reality Converter / usdzconvert / online	Adds iPhone AR support
Optimize model	glTF-transform, Draco, meshopt, glTF Report	Small file = fast, no lag
Fix / create model	Blender, Meshy (AI), Polycam / Luma (scan)	Better looking, correct size
The page	Plain HTML / CSS / JS (no React needed)	Simplest possible MVP
Hosting	Netlify / Vercel / GitHub Pages / Cloudflare	Free + automatic HTTPS
Entry point	QR-code generator	Jump from a laptop/poster to a phone

Two non-negotiables

- **HTTPS is mandatory** — the camera and AR silently fail on a plain `http://` link.
- **Provide both files** — GLB for Android/web and USDZ for iPhone, or iPhone users get no AR.

SECTION 10

Milestones (a clear path to done)

#	Milestone	Outcome / how we know it's done
M0	Setup	Project folder + a blank page scaffold + a chosen placeholder GLB.
M1	3D viewer	The washing machine shows on desktop; you can rotate and zoom it.
M2	Android AR	"View in your space" opens Scene Viewer/WebXR; machine sits on the floor at correct size.
M3	iPhone AR	GLB converted to USDZ; Quick Look works on iPhone at correct scale.
M4	Polish	IFB branding, dimensions readout, variant switch, CTA, loading poster, error/fallback states.
M5	Deploy + QA	Live HTTPS link + QR; tested on Android, iPhone, tablet and laptop.
M6	Real model (optional)	Swap in the optimized IFB-scanned model; add analytics / more products.

MVP = M0 through M5. M6 makes it production-grade with the real product.

SECTION 11

Blockers & risks (and how we beat each one)

Risk / blocker	Why it matters	How we handle it
iPhone has no in-browser AR	Live AR won't run in Safari on iPhone	Ship a USDZ and use Quick Look; verify materials after conversion.
Model floats or sinks	Looks broken; not "on the surface"	Set the origin at the base-centre ; use <code>ar-placement=floor</code> .
Wrong size	Machine appears tiny or huge	Build at real metres (1 unit = 1 m) + <code>ar-scale=fixed</code> .
Big file = lag	Slow load / stutter on mobile data	Compress (Draco/meshopt); target < ~5–10 MB.
Not on HTTPS	Camera / AR silently fail	Deploy on an HTTPS host (Netlify/Vercel).
Old / unsupported device	No ARCore or ARKit	Graceful 3D-only fallback + a friendly message.
In-app browsers	Instagram/Facebook webviews block AR	Detect and prompt "open in Chrome/Safari".
Realism limits	Model won't hide behind real furniture	Set expectations; add 8th Wall later only if truly needed.
Poor placeholder look	Rough open-source model	Replace with a Sketchfab CC model or a real IFB scan.

SECTION 12

2026 trends & sources

Where Web-AR is heading (why our choice is future-safe)

- **App-less Web-AR is winning for retail** — Shopify and Amazon push GLB + USDZ straight from the browser; shoppers won't download an app just to view one product.
- **<model-viewer> is still the default** for “put a product in your room” — minimal code, maximal reach.
- **8th Wall went free / open-source (Feb 2026)** — SLAM tracking is now within reach if we ever need portals, occlusion, or marketing games.
- **AI is fixing the hard part — the model:** tools like **Meshy** (text/image → 3D) and **Luma / Polycam** (phone scanning) make a good washing-machine model cheap and fast.
- **iPhone still gates in-browser AR** in 2026 — so GLB + USDZ (our plan) remains the correct cross-platform path.

Sources (clickable)

model-viewer — AR docs (Google)	https://developers.google.com/ar/develop/webxr/model-viewer
model-viewer official site	https://modelviewer.dev/
Google Scene Viewer (Android AR)	https://developers.google.com/ar/develop/scene-viewer
Amazon — 3D & AR for products	https://sell.amazon.com/tools/3d-ar
IKEA + ARKit + Amazon (retail AR)	https://3dcloud.com/ikea-arkit-amazon-and-6-keys-to-winning-with-augmented-reality-for-retail/
8th Wall goes open-source (Road to VR)	https://roadtovr.com/niantic-webar-platform-8th-wall-open-source/
WebXR on iOS — what works in 2026	https://xrdoctors.pro/blog/webxr-on-ios-what-actually-works
Best 3D format for AR (GLB vs USDZ)	https://geometryviewer.com/blog/best-format-for-ar
Meshy — AI text/image → 3D	https://www.meshy.ai/
awesome-WebAR (curated tools list)	https://github.com/tobiasbueschel/awesome-WebAR

Next step

Keep or replace the placeholder model, and we start building the microsite (M0–M5): a single `index.html` with `<model-viewer>`, the GLB + USDZ, a dimensions readout, and a one-click deploy to an HTTPS link with a QR code.